

Email Service

Overview

The Email Service is a microservice responsible for processing email events in an event-driven architecture. It consumes messages from RabbitMQ, generates email content using AI, and sends emails via SMTP. The service also ensures idempotent processing by storing handled events in a MySQL database.

Architecture

The service follows a message-driven architecture:

1. Messages are published to RabbitMQ (from other services)
 2. Email Service consumes messages from a queue
 3. Messages are wrapped in an **event envelope**
 4. The payload is processed and transformed into an email
 5. Email content is optionally enhanced using AI
 6. The email is sent via SMTP (Mailjet)
 7. The event is stored in the database to prevent duplicate processing
-

Event Envelope

All incoming messages follow this structure:

```
{
  "eventId": "uuid",
  "eventType": "EmailRequested",
  "payload": {
    "to": "user@email.com",
    "subject": "Subject text",
    "body": "Email content"
  }
}
```

Purpose of the envelope:

- Enables **idempotency** (via `eventId`)
 - Allows tracking and auditing of events
 - Supports extensibility across services
-

Features

Email Processing

- Deserializes incoming messages
- Validates payload
- Sends emails using SMTP

AI Email Generation

- Uses external AI API (OpenRouter)
- Enhances email body content
- Falls back to original message if AI fails

Idempotency

- Stores processed events in MySQL
- Prevents duplicate email sending

Database (MySQL)

- Table: `ProcessedEvents`
- Tracks:
 - `eventId`
 - `eventType`
 - `processedAt`

Seeder

- Automatically inserts sample data on startup
- Simulates historical event processing

Docker Setup

The service runs as part of a Docker Compose setup.

Services:

- Email Service (custom Dockerfile)
- MySQL (official image)
- RabbitMQ (message broker)

Start system:

```
docker compose up --build
```

Environment Variables

Defined in `.env`:

```
MYSQL_CONNECTION=server=mysql;port=3306;database=emaildb;user=root;password=1234

MYSQL_ROOT_PASSWORD=root
MYSQL_DATABASE=emaildb
MYSQL_USER=root
MYSQL_PASSWORD=1234

OPENROUTER_API_KEY=your_api_key_here
MAILJET_API_KEY=your_mailjet_key
MAILJET_SECRET=your_mailjet_secret
```

Testing

The service includes unit tests using:

- xUnit
- Moq

Test coverage includes:

- Email sending logic
- AI fallback behavior
- Input validation
- Edge cases

Run tests:

```
dotnet test
```

Flow Summary

```
RabbitMQ → Worker → Deserialize Envelope → Check DB → Generate Email (AI) →  
Send Email → Save Event
```

Design Decisions

Event-Driven Architecture

- Decouples services
- Improves scalability
- Enables asynchronous communication

Envelope Pattern

- Standardizes message structure
- Supports tracking and debugging

Idempotency

- Prevents duplicate processing
 - Critical in distributed systems
-

Future Improvements

- Add Dead Letter Queue (DLQ)
 - Implement retry logic
 - Add health checks
 - Improve logging and monitoring
 - Replace `EnsureCreated()` with migrations
-

Author Notes

This service is part of a microservices-based system using:

- .NET
- RabbitMQ
- MySQL
- Docker

It demonstrates real-world patterns such as:

- Event-driven communication
 - Idempotent processing
 - Service isolation
-